

13. Deep Learning 6

Tips and Tricks

Machine learning

Jiri Chmelik

Department of Biomedical Engineering

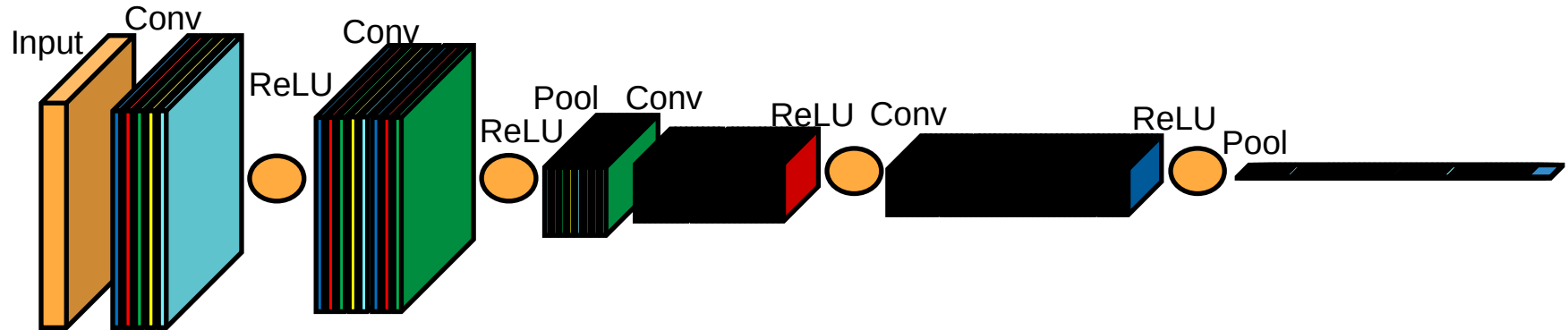
Introduction

- Architectures
- Tips and tricks

Architectures

Standard encoder

- Set of various layers in series, which extracts a tensor of features (usually $1 \times 1 \times \text{Depth}$), which enough to credibly describe input data for classification/regression problem
- Data size is gradually reduced in the Width and Height directions, while the dimension in Depth is growing
- The output is tensor (1D vector) of relevant features coding the input data (encoder)



Alexnet (2012)

- Only series connection of five convolution layers with three FC layers

Includes:

- ReLU nonlinearity,
- Overlapping Pooling
- Dropout (50%)
- Cross Channel Normalization
- Total 25 layers

Pros.

- Simple architecture
- Fast learning and prediction

Cons.

- architecture does not has BatchNorm
- Low capacity for complex problems

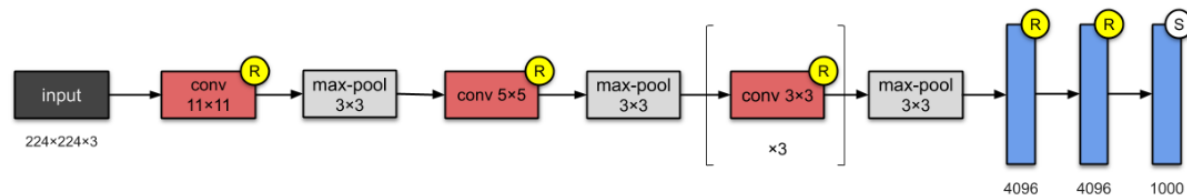


Fig. 2: AlexNet architecture, based on their [paper](#).

VGG-19 (16)

- Only series connection of 16 convolution layers and 3 FC layers
- The Use of 3×3 Filters instead of large-size filters

Includes:

- 16 conv 3×3
- “Zero-center” normalization
- Total 47 layers

Pros.

- deeper architecture than Alexnet
- Still simple architecture (series)

Cons.

- Large size (over 500MB)
- Too much parameters
- No BatchNorm

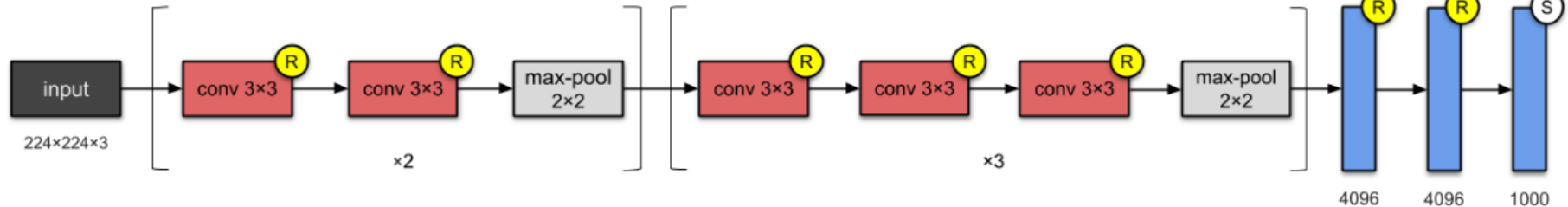
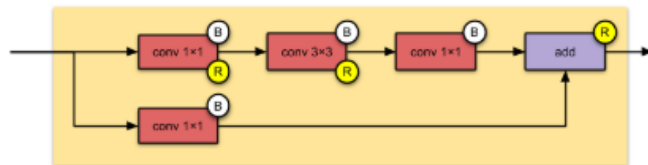
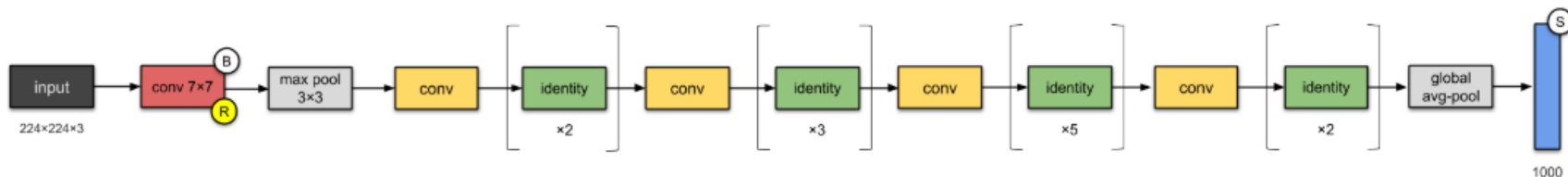


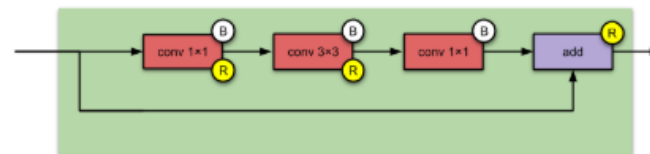
Fig. 3: VGG-16 architecture, based on their [paper](#).

ResNet (layers – 18,50,101,...)

- Contains residual blocks (skip connections) and bottleneck layers
- The basic building blocks for ResNets are the conv and identity blocks
- Very deep CNN architecture (up to 152 layers)
- Solves the problem of vanishing gradients
- Uses BatchNorm
- Newer versions use additional connections in parallel branches



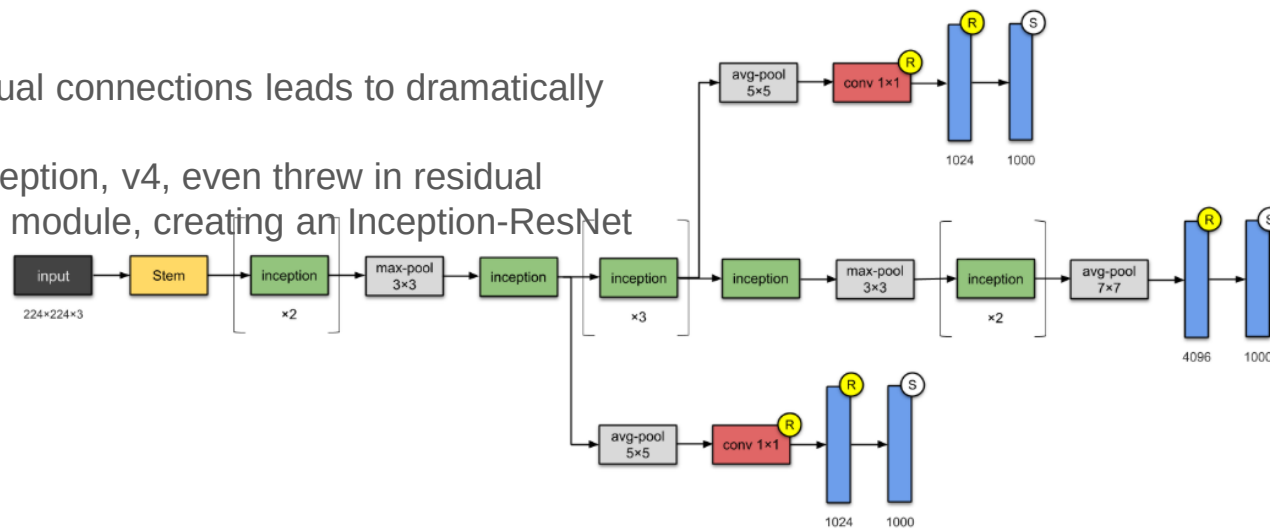
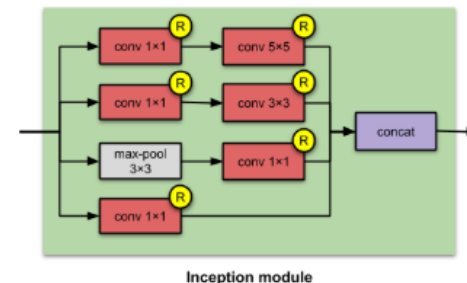
Conv block



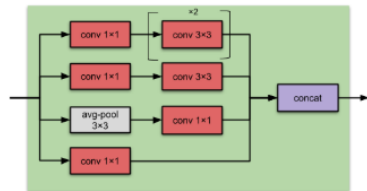
Identity block

Inception V1-4 (2014-2016)

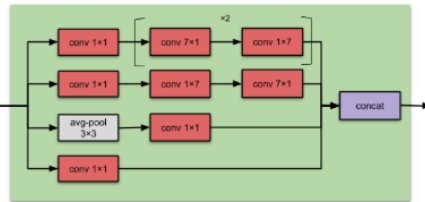
- Introduced Inception module (parallel block of bottlenecks)
 - Computes multiple different transformations over the same input map in parallel, concatenating into a single output
 - Two auxiliary classifiers for keep gradients
 - Separable (asymmetric) convolution $1 \times n$, $n \times 1$ (faster)
 - Convolution 3×3 instead of $n \times n$
 - Uses BatchNorm
- V4 contains newly residual connections leads to dramatically improved training speed
 - The latest version of Inception, v4, even threw in residual connections within each module, creating an Inception-ResNet hybrid.



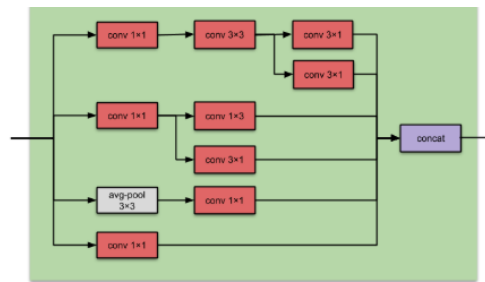
Inception v4 (2017)



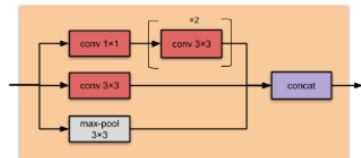
Inception-A



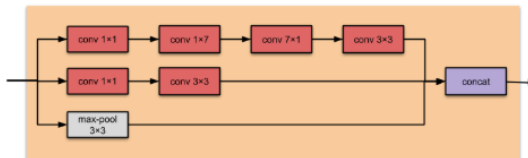
Inception-B



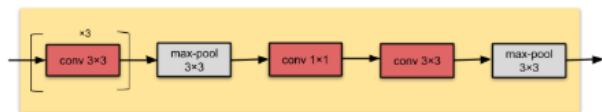
Inception-C



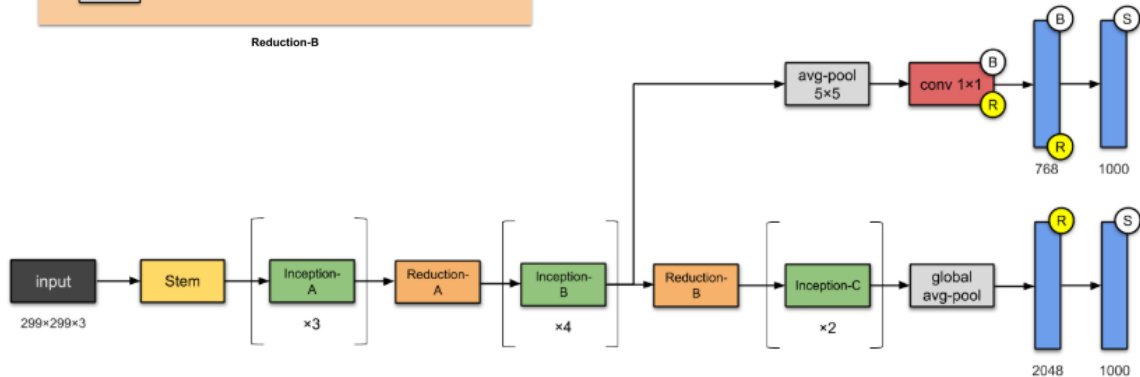
Reduction-A



Reduction-B



Stem



ResNetXt 50 (2017)

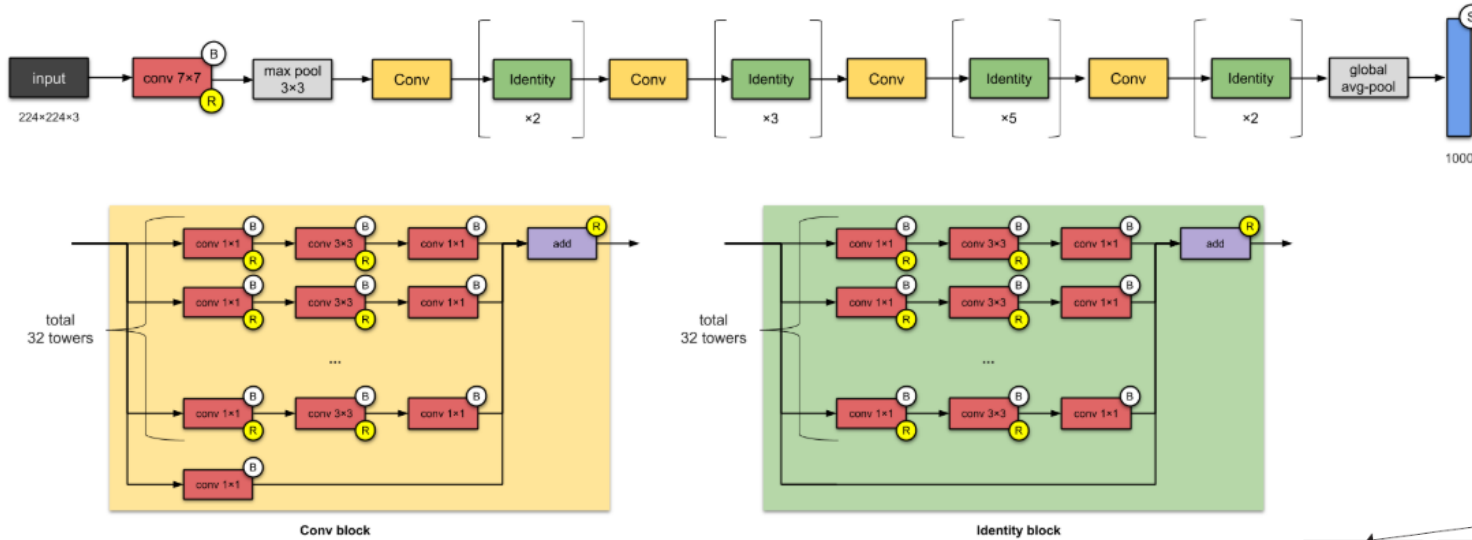
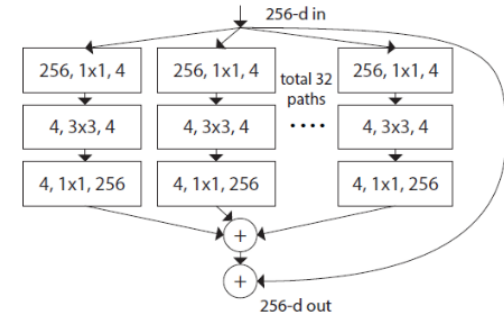
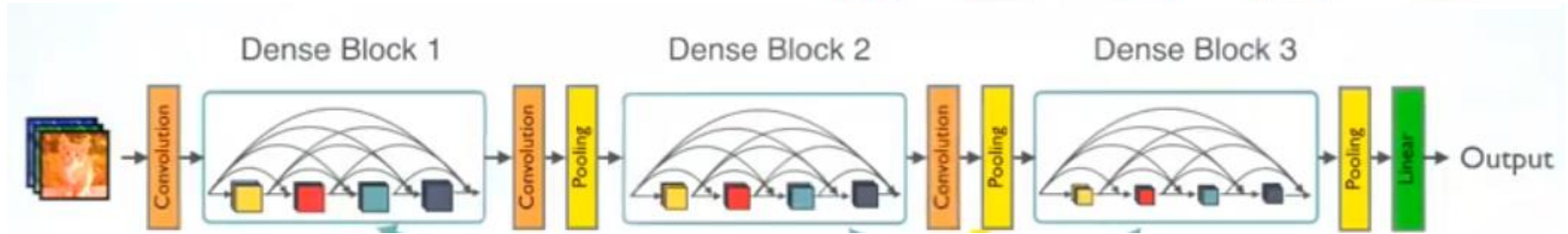
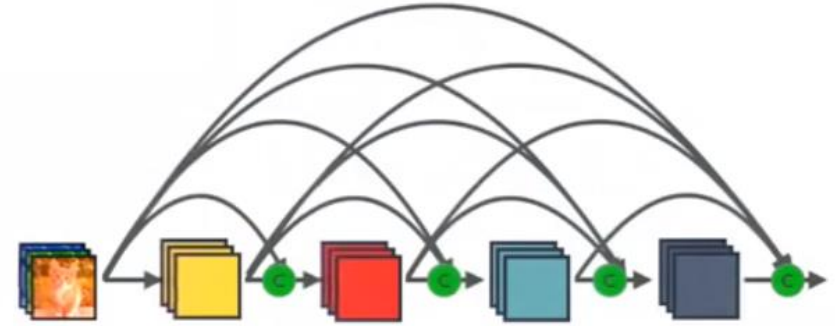


Fig. 10: ResNetXt architecture, based on their paper.

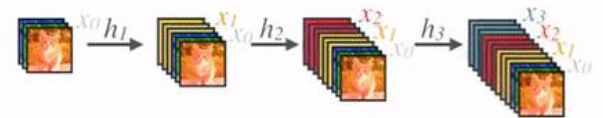


DenseNet

- Special short connections with concatenation
- Over 700 layers



- classifier uses features of all complexity levels => performs well when training data is insufficient.
- Error signal can be propagated to earlier layers, thus earlier layers can get direct supervision from the final classification layer

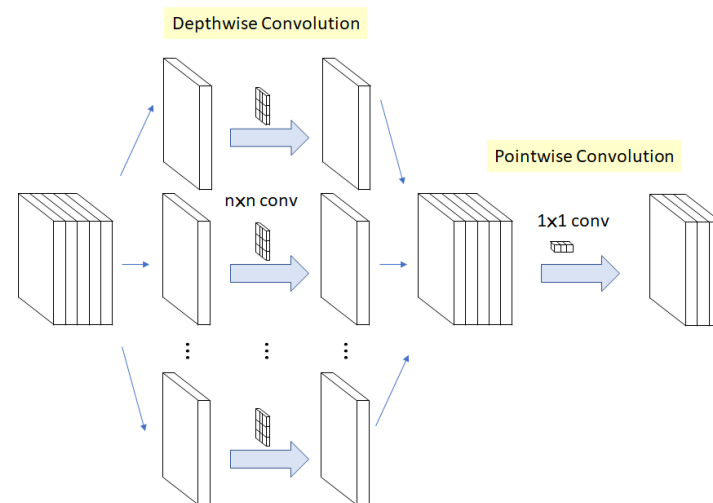


MobileNets

- Depth-wise separable convolution
- 2D convolution (spatial) for each feature maps followed by 1x1 convolution in depth (called depth-wise and point-wise convolutions)
- very fast approach and reduced model size
- for Mobile Vision Applications
- Object detection and classification in real time

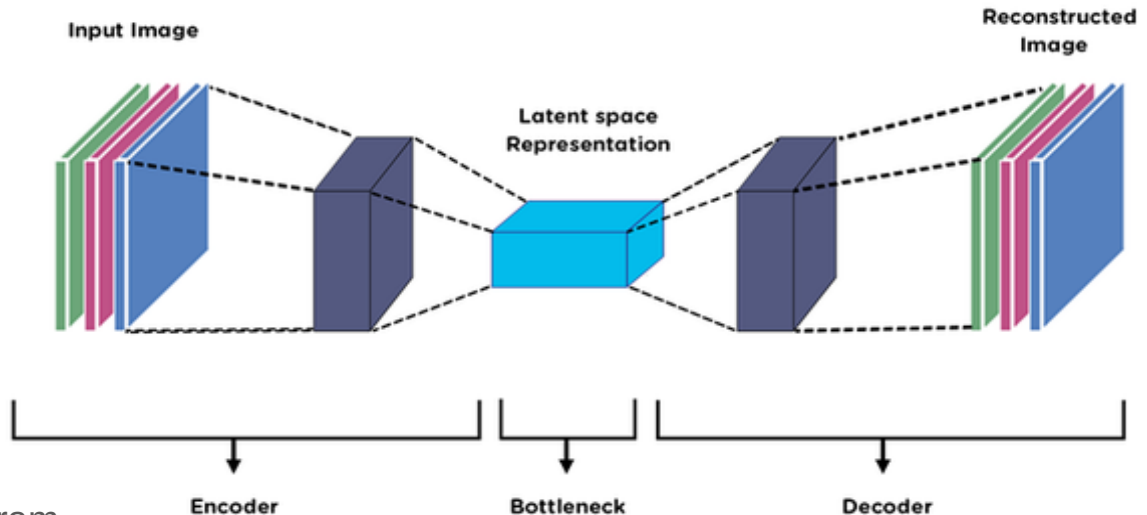
Table 8. MobileNet Comparison to Popular Models

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
1.0 MobileNet-224	70.6%	569	4.2
GoogleNet	69.8%	1550	6.8
VGG 16	71.5%	15300	138
0.50 MobileNet-160	60.2%	76	1.32
Squeezenet	57.5%	1700	1.25
AlexNet	57.2%	720	60



Pair encoder – decoder

- The architecture is based on idea of bottleneck
- Encoder codes image into feature tensor using conv layers and max-pooling layers
- Decoder decodes (reconstructs) tensor into new image by transposed convn and un-pooling
- Input: images
- Output: new image
- “image-to-image”

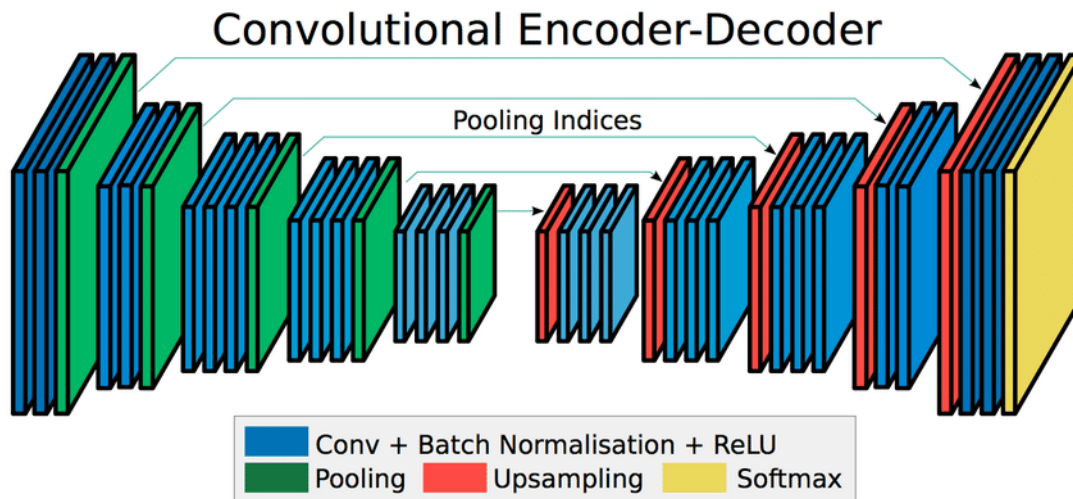


Utilizing:

- Noise reduction
- Image segmentation
- Reconstruction from sinogram
- Transformation input image to a new image

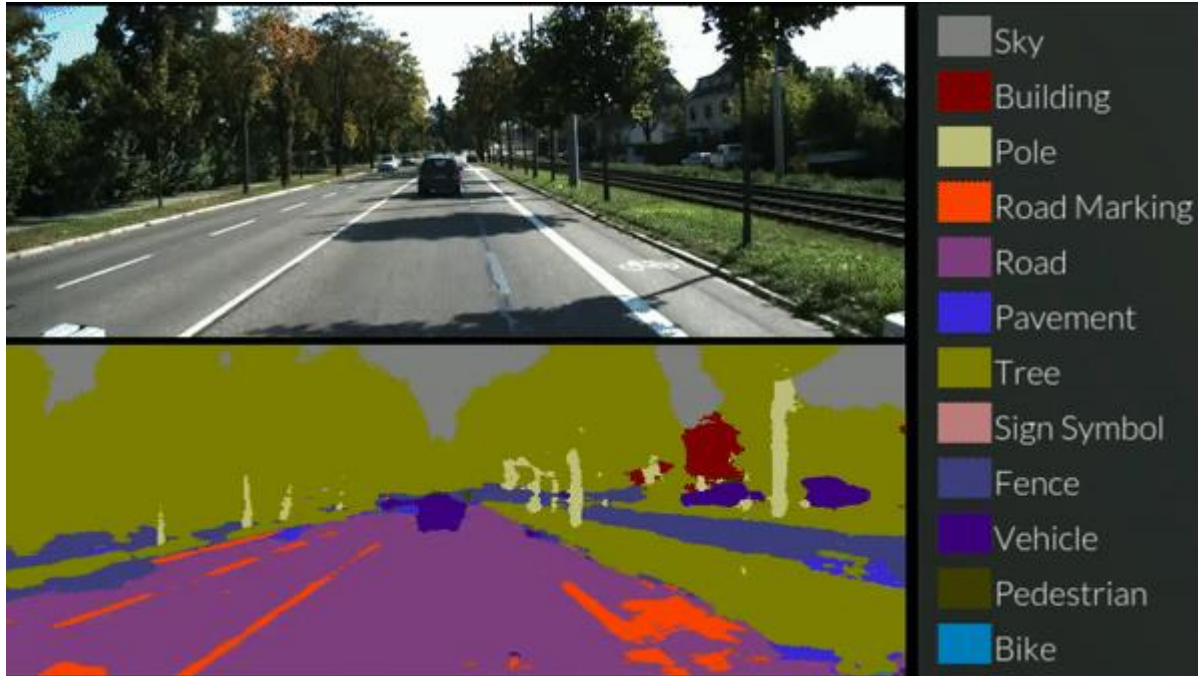
SegNet (2015)

- Architecture: encoder – decoder with bottleneck
- For up-sampling, the architecture uses un-pooling with shared indices through long connections
- Better than interpolation, but it provides still imprecise borders of objects
- In the end, there is a softmax classifier for each pixel.



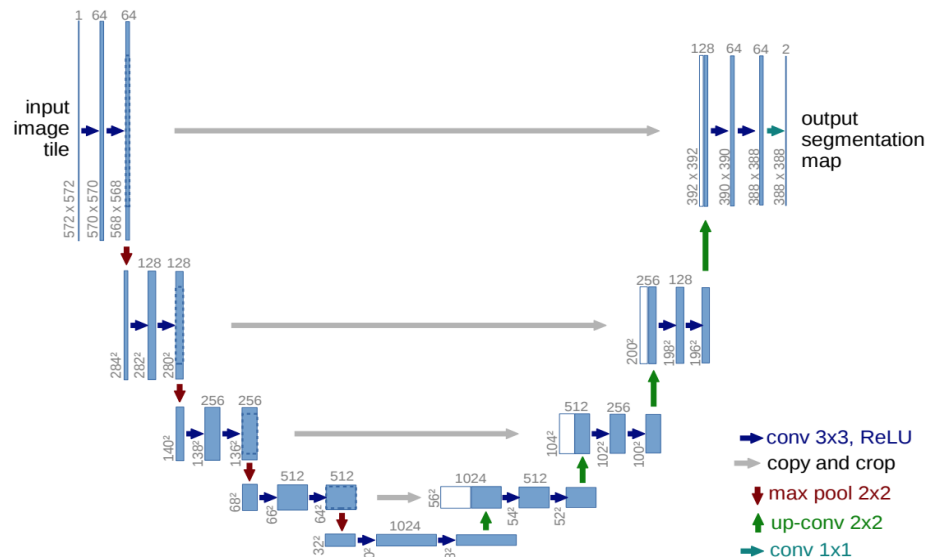
SegNet

- Example



U-Net (2015)

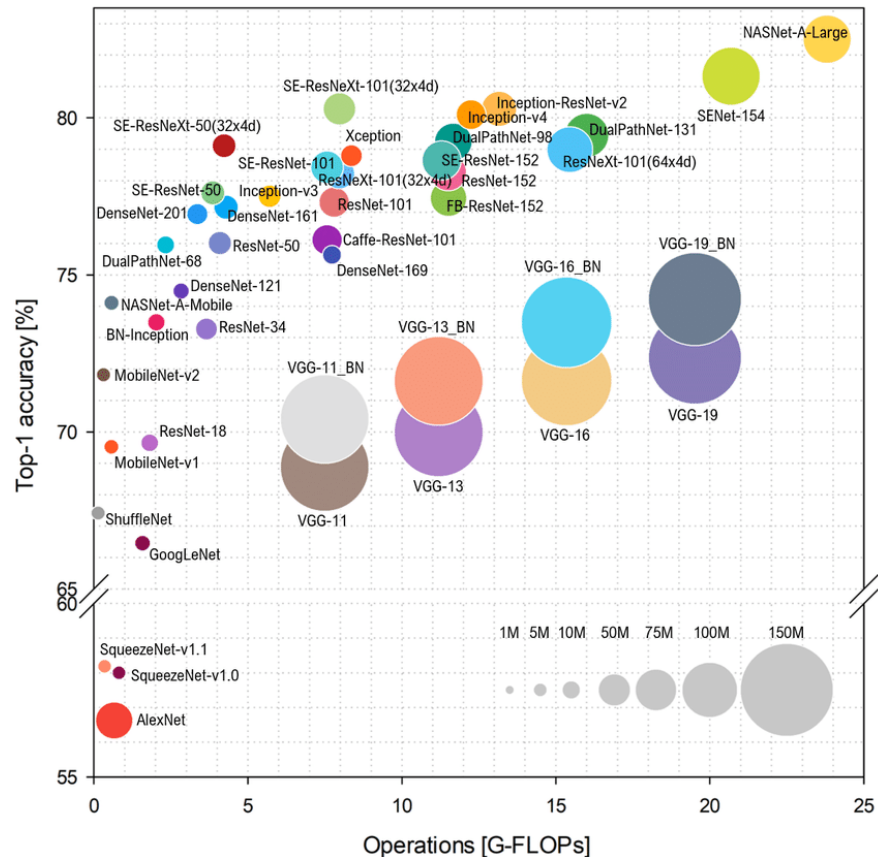
- Developed for biomedical image segmentation
- Use long **skip connections** between the down-sampling path and the up-sampling path
- Skip connections with **concatenation**
- The bottleneck is built from two 3x1 convolutional layers (with BN and ReLu)
- It combines the location information from the down-sampling path with the contextual information in the up-sampling path
- No general pre-trained model
- There are many pre-trained models for specific segmentation problems (e.g. MRI brain)
- But, we can use e.g. VGG pre-trained model in the encoder part



Other architectures

- SqueezeNet
- DualPathNet
- SENet
- ShuffleNet
- NASNet
- SegNet
- FCNs – fully convolution network
- V-Net – volumetric segmentation for biomedical images
- N-Net – for vertebra segmentation
- Btrfly Net – for vertebra identification
- DeepLab
- ParseNet
- FPN – feature Pyramid Network
- PSPNet – Pyramid Scene Parsing Network
- Cascaded 3D U-Net – 3D for multi-organ segmentation
- PANet – Path Aggregation Network (2018)
- EncNet – Context Encoding Network (2018)
- ...

Architecture Comparison

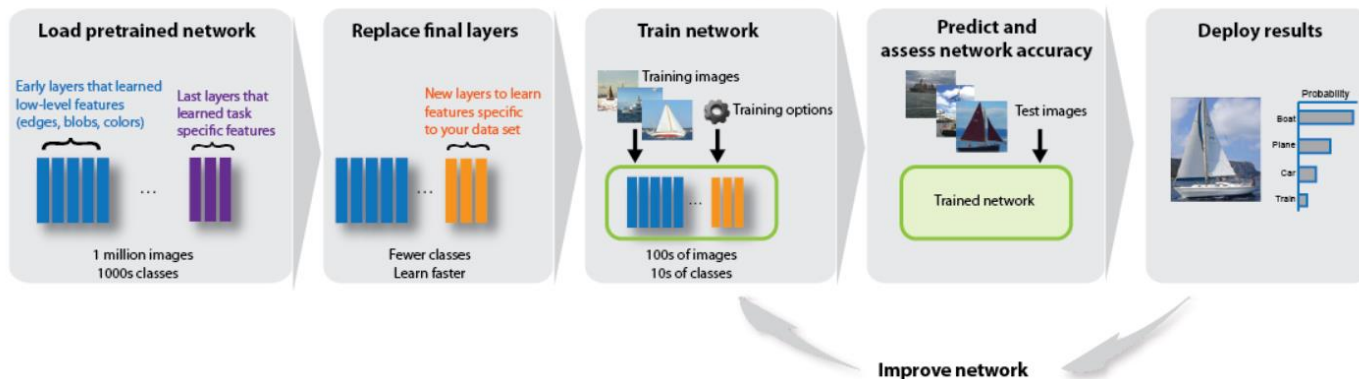


Tips and Tricks

Transfer learning

- It turned out, that early layers have a tendency converge to impulse responses corresponding the edge detectors (first and second derivation, edge detectors, Gabbors' or Haars' masks).
- These basic features are then propagated to following layers, and there are combining into complex (abstract) features.
- Is the process of taking a pre-trained model (the weights and parameters of a network) and fine-tuning the model with our own dataset

Reuse Pretrained Network



Pre-trained networks

- most standard publicly available architectures (Alexnet, ResNet, Inception,...)
- Pre-trained on ImageNet dataset
- classification task – to assign into 1000 categories

- We use a same architecture, and we utilize weights of neurons as initial values for fine-tuned network on our dataset

Cons.

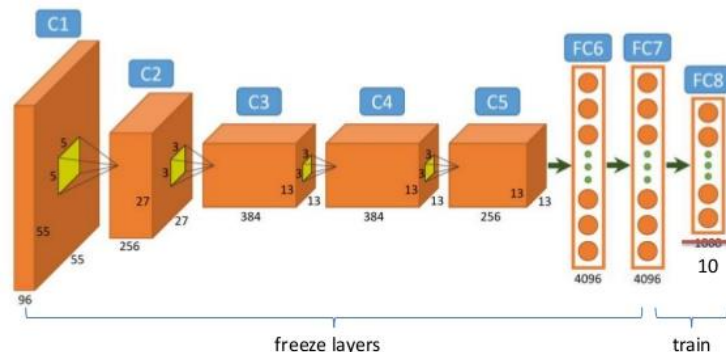
- We must use the same architecture, such as was pre-trained
- The solution to our problem can be other (e.g. more complicated or non standard,...)

Pros.

- The training process requires lower computed time
- Solution more complex problems (with local extreme in cost function)
- Avoid overfitting

Freeze approach

- use a network trained on unrelated categories in a massive dataset (usually Imagenet) and apply it to our own problem because there are universal, low-level features shared between images
- The idea is the convolutional layers extract general, low-level features that are applicable across images — such as edges, patterns, gradients — and the later layers identify specific features within an image such as eyes or wheels.
- We can **freeze** the early convolutional layers of the network and only train the last few layers which make a prediction
- n early layers can be frozen or fine-tuned
- **Frozen** – not updated during backprop.
- **Fine-tuned** – updated during backprop.



ImageNet

- is image dataset organized according to the WordNet hierarchy
- there is provided on average 1000 images to illustrate each synset
- Total number of non-empty synsets: 21,841
- Total number of images: 14,197,122
- Number of images with bounding box annotations: 1,034,908

Statistics of high level categories

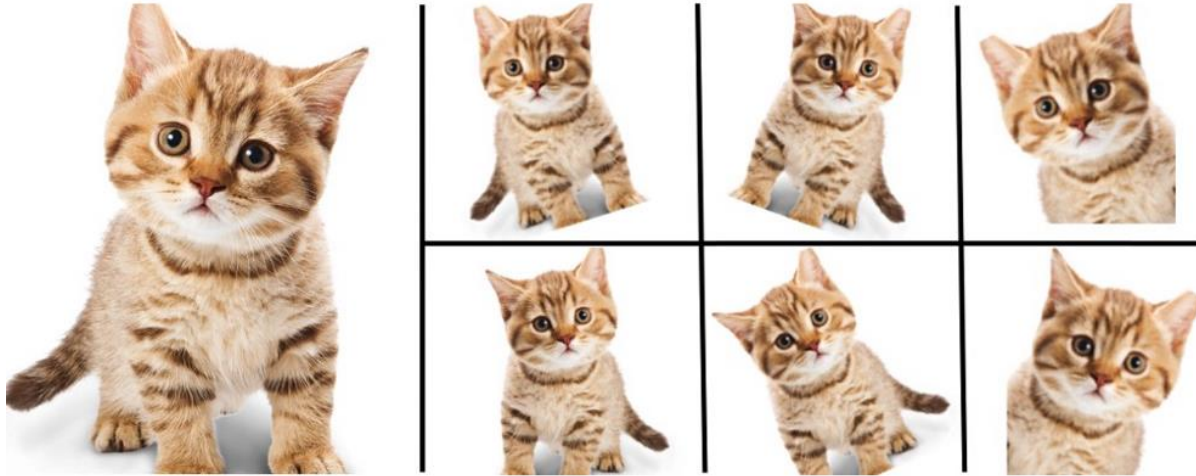
High level category	# synset (subcategories)	Avg # images per synset	Total # images
amphibian	94	591	56K
animal	3822	732	2799K
appliance	51	1164	59K
...	...	...	...

- Most often, the 1000 categories is used for machine learning

Data augmentation

- It is the process for extending the training dataset, which leads to more robust NN.
- We create synthetic variants (synthetically modified) from real images, which can occur in the real world, but it is not available in our dataset.
- **Types:**
 - geometrical transformation (flip, rotation, scaling, crop, ...)
 - intensity modification (point-wise operation, color,)
 - noising
 - blurring/sharpening
 - data mixing
 - random image generation (GAN)
- **Realization:**
 - **offline approach** (smaller datasets) before we feed the data to the model.
 - **online approach** is augmentation in the fly, the transformations are performed on the mini-batches (GPU).

Data augmentation



Enlarge your Dataset

End-to-end learning

- Solution, where are many inputs from different sensors (multimodality data) and/or many tasks to solve (e.g., classification, segmentation and regression of different quantities).
- It uses a single complex model for all tasks.
- A huge amount of data is needed.
- Fine-tuning or re-training the model is difficult or even impossible.
- Needs a complex and balanced training approach (e.g., one problem is solved while solution of another is not sufficient – trying to improve the results of second task may lead to worsening of the first one).

Frameworks

Review

- TensorFlow 1 (Google)
 - interface for other languages
 - static computation graph
 - for Mobile application (Android, iOS) or webpages
 - it strictly requires the design of architecture, changes cause the need for retraining
 - more complex code, but better-solved deployment
- TensorFlow 2 (Google) (+ advantages of PyTorch)
- Keras
 - for beginners
 - the easiest frameworks
 - simple concepts
 - little and readable code
- PyTorch
 - simplicity, flexibility, and customizability
 - for prototyping and researchers
 - you can use a debugging
 - dynamic computation graph – changing of architectures in the process



Frameworks

Application

Research purposes – PyTorch, TensorFlow2

For beginners – Keras

For products, commercial utilizing – TensorFlow, (MXNet, Gluon)

Transfer deep learning models – ONNX

For Android developers – D4LJ

For iOS developers – Core ML

